AIStudio

HOWTO — AIStudio User Guide

Practical answers for day-to-day AIStudio use. Not a getting-started guide (see QUICKSTART.md) — reach for this when you need to do something specific or something isn't working as expected.

Version: Beta | Updated: 2026-07-03

Contents

Shell & Terminal

What is the shell? The shell (Terminal) is a text-based interface for running commands on your Mac. AIStudio setup and management uses the shell for tasks the UI doesn't cover — like installing models or ingesting a new corpus. New to the shell? See Apple's Terminal User Guide.

How do I open the Terminal? Press `Cmd+Space`, type `Terminal`, press Enter. Or open Finder → Applications → Utilities → Terminal.

Why does my terminal say "command not found" when I paste commands? Two common causes: - You pasted a `#` comment line — zsh tries to execute it. Paste only the commands, not the comment lines. - Your AIStudio aliases aren't loaded yet. Run:

```
source ~/.zshrc
```

Then try the command again on its own line. Do not chain: `source ~/.zshrc && ais_start` will fail. Run `source ~/.zshrc` first, then `ais_start`.

Using AIStudio

After running `./ais_install` from the repo root (see QUICKSTART.md), these commands are available from any terminal. If a command is not found, run `source ~/.zshrc` first.

Command	What it does
`ais_install [cmd]`	Install AIStudio user commands — `ais_install ais_log` adds a single command, `ais_install --verify` checks all aliases
`ais_start`	Start all services and open the UI in your browser
`ais_stop`	Stop all services
`ais_bench`	Run a benchmark on the demo corpus
`ais_log`	Tail live AIStudio backend log — run in a separate tab after ais_start
`ais_download_sec_10k`	Download SEC 10-K filings from EDGAR into the corpus (`data/corpora/sec_10k/uploads/` ; ~900 MB for the full multi-year set)
`ais_help`	Print this command reference

Every command supports `--help`:

```
ais_start --help
ais_bench --help
```

Upgrading AIStudio

How do I upgrade AIStudio when a new version is released?

Pull the latest code from GitHub:

```
cd ~/Developer/AIStudio && git pull
```

Then update Python dependencies in case new packages were added:

```
source .venv/bin/activate && pip install -r requirements.txt
```

Then restart:

```
ais_start
```

Does upgrading delete my corpus data? No. Your corpus data lives in `~/qdrant_storage/` on disk — completely separate from the AIStudio codebase. It persists across upgrades, restarts, and reboots. You never need to re-ingest after a routine upgrade.

When would I need to re-ingest after an upgrade? Only if the release notes say the chunk format changed. This is rare and always announced. When it happens, re-ingest from the UI: open the corpus, **Delete Corpus**, recreate it with the same name, and re-upload your files with **Add** — AIStudio rebuilds it cleanly in one pass. There is no **Force** button in the UI — that Delete-and-recreate flow *is* the clean rebuild, and it's all you need. (A `--force` flag exists only for operators re-ingesting from the terminal; it is not part of the UI workflow.)

The demo corpus re-ingests automatically on first `ais_start` after an upgrade if needed.

Corpus Management

How do I ingest a new corpus? Use the UI — open AIStudio, create a new corpus using the **New** button, then upload your files using the **Add** button. AIStudio handles ingestion automatically and shows progress in the chat area.

How do I re-ingest a corpus from scratch? Delete the corpus using the **Delete Corpus** button in the UI (type YES to confirm — it moves to Trash, recoverable). Then create a new corpus with the same name and re-upload your files via **Add**. AIStudio ingests everything in one pass.

How do I delete a file from a corpus? Use the UI — select your corpus, click **Inspect** to open the file list, then click **Delete** next to the file you want to remove. AIStudio moves the file to trash and removes its chunks from the vector index. You do not need to re-ingest the rest of the corpus — only the deleted file's chunks are removed.

What happens when I delete a file from a corpus — is it gone forever? No. Deleted files move to `data/corpora/<name>/uploads/trash/` — not permanently deleted.

How do I recover a file I accidentally deleted from a corpus?

```
# See what's in trash
ls ~/Developer/AIStudio/data/corpora/<name>/uploads/trash/
```

```
# Move it back
mv ~/Developer/AIStudio/data/corpora/<name>/uploads/trash/<filename> \
   ~/Developer/AIStudio/data/corpora/<name>/uploads/

# Then re-upload the file via the UI Add button to restore it
```

What happens when I delete an entire corpus — is it gone forever? No. The corpus folder moves to `~/.Trash/`. The folder is renamed to avoid conflicts with previous deletions — it will be named `AIStudio_<name>` or `AIStudio_<name>_<timestamp>` if a folder with that name already exists in Trash.

How do I recover a corpus I accidentally deleted? First find the folder in Trash — it may have a timestamp suffix:

```
ls ~/.Trash/ | grep AIStudio_<name>
```

Then move it back and re-ingest:

```
mv ~/.Trash/AIStudio_<name> ~/Developer/AIStudio/data/corpora/<name>
# Or if it has a timestamp suffix:
mv ~/.Trash/AIStudio_<name>_<timestamp> ~/Developer/AIStudio/data/corpora/<name>
# Then re-ingest
```

Then re-upload the files via the UI (Add button) to restore the corpus in Qdrant.

What if the Trash folder is gone (emptied), or I deleted a built-in corpus like SEC 10-K or ESEF banks? The built-in corpora (`sec_10k` , `esef_banks` , plus `demo` and `help`) keep their *recipe* files — the scope and configuration that define the corpus — in the AIStudio repository itself, so you can always get those back even if Trash is gone:

```
cd ~/Developer/AIStudio
git checkout -- data/corpora/<name>/      # restores the corpus's scope + config files
```

That restores the **recipe** (what the corpus is). To rebuild the **data** (the filings and the searchable index), follow the corpus's normal build steps — for SEC 10-K / ESEF banks that's the download → ingest flow in **TUTORIAL Module 2 / 3**; for `demo` and `help` the files ship with the repo, so a `git checkout` of the folder plus a re-ingest is enough. (Operators: the full delete/restore runbook, including how to verify nothing is missing, is in HOWTO_OPS.)

How do I ingest the SEC 10-K corpus?

The SEC 10-K corpus is **not** a one-line recipe, and — unlike a corpus you bring yourself — the ingest does real corpus-specific work. The full walkthrough lives in **TUTORIAL Module 2**; here's the shape so you know what's involved:

1. **Download** — `ais_download_sec_10k` pulls the filings from SEC EDGAR directly into the corpus (`data/corpora/sec_10k/uploads/`; ~900 MB for the full multi-year set). EDGAR's access protocol is why this step is automated for you.
2. **Entity knowledge base** — before ingest, the corpus is given a verified entity KB (LEIs + aliases) so retrieval can bind each chunk to the right firm. (TUTORIAL §2.2.)
3. **Glossary** — a Basel-vocabulary glossary for query-time term expansion. (TUTORIAL §2.3.)
4. **Ingest** — this is where chunk **enrichment** happens: each iXBRL filing is read for its entity and fiscal year, and every chunk is prefixed with `[Document: <entity> FY<year>]`. That prefix is what lets a terse table row be attributed to the correct firm and year, and it's exactly why this corpus is **not** "the same as ingesting any corpus you bring yourself."

So the special part isn't only the download — it's the entity/year enrichment and the knowledge bases that go with it. See **TUTORIAL Module 2** for the step-by-step.

Installing and Managing LLMs

How do I see what models are installed?

```
ollama list
```

How do I install a new LLM?

```
ollama pull gemma3:4b          # ~3.3GB — the out-of-the-box default, fast
ollama pull gemma3:27b         # ~17GB — best quality-for-size; the benchmark model
ollama pull llama3.1:8b        # ~5GB — alternative, fast
ollama pull llama3.1:70b       # ~42GB — best quality, requires ~64GB RAM
ollama pull mistral:7b         # ~4.4GB — alternative option
```

Once pulled, the model appears automatically in the **Model** dropdown in the UI. No restart required.

Which model should I choose? On Apple Silicon, `llama3.1:70b` and `llama3.1:8b` have identical query latency (~6–7s) once warm. Choose based on available RAM: - `gemma3:4b` — 8GB+ RAM, the out-of-the-box default; small, fast, best first run - `gemma3:27b` — 32GB+ RAM, best quality-for-size; the model AIStudio benchmarks on - `llama3.1:8b` — 8GB RAM minimum,

solid alternative - `llama3.1:70b` — 64GB+ RAM, best answer quality - `mistral:7b` — good alternative on constrained hardware

How do I remove a model I no longer need?

```
ollama rm mistral:7b
```

See the Ollama model library for all available models.

Query Settings

The **Query Settings** panel in the sidebar controls how AIStudio retrieves and generates answers. These settings apply to every query until you change them.

Top K — *How many document chunks to retrieve* The number of passages retrieved from your corpus before generating an answer. Higher values give the model more context but increase latency slightly. - Default: `5` — good for most queries - Try `10` for complex questions spanning multiple documents - Try `3` for faster responses on focused questions

Choosing Top K by query type:

Query type	Recommended Top K	Example
Point-in-time, single firm	5	"What was Goldman's CET1 ratio in 2024?"
Trend across multiple years	10–15	"How has JPMorgan's capital ratio changed 2022–2025?"
Cross-firm comparison	10	"Compare risk disclosures at JPMorgan, Goldman, and Citi"
General question	5	"What is the company's AI strategy?"

If an answer seems incomplete or missing years/firms you expected — increase Top K first before rephrasing the question.

Adjust using the **Top K** input field in the sidebar.

Temperature — *How creative vs. precise the answer is* Controls randomness in the model's output. - `0.1–0.3` — precise, factual, stays close to the source documents. Best for financial data, specific numbers, compliance questions. - `0.5–0.7` — more varied, synthesizes across

sources. Better for summaries. - Default: `0.3` — recommended starting point Adjust using the **Temperature** input field in the sidebar.

Retrieval Mix — *Balance between literal and conceptual search* Controls how the system finds relevant passages in your corpus.

- **Literal** (slider left, value 0.0) — prioritizes exact word and phrase matching (BM25). Use when you know the specific term, entity name, or abbreviation that appears in your documents. Example: searching for "CET1" by exact term.
- **Conceptual** (slider right, value 1.0) — prioritizes semantic meaning. Finds passages related to your query even when the document uses different phrasing. Example: asking about "capital strength" returns passages about CET1, Tier 1 capital, and capital ratios.
- **Default 0.5** — blends both. Works well for most queries.

Choosing by query type:

Query type	Recommended setting	Why
Named entity, ticker, exact term	Literal (0.0–0.3)	BM25 matches the token directly
Thematic or conceptual question	Conceptual (0.7–1.0)	Semantic search handles paraphrase
Multi-firm comparison	Center (0.4–0.6)	Entity names + topic meaning both matter
Single firm, multiple years	Conceptual (0.8–1.0)	Year context is in documents; meaning guides retrieval

Adjust using the **Retrieval Mix** slider in the Query Settings sidebar.

Score Threshold — *Relevance floor* Chunks scoring below this are dropped before the model ever sees them. Default `0.50`. Lower it (≈`0.2–0.3`) if good answers come back empty or hedged; raise it to cut noise. Set it **per query** in the **Threshold** field in the sidebar, or **per corpus** in **Edit** → **Query Defaults** (where it applies whenever that corpus is queried). The per-query field wins; leave it blank and the corpus default applies.

Timeout — *How long to wait for the model* The maximum time AIStudio waits for the model to answer before giving up. Default `300` seconds — generous on purpose, so a slow large model (a 70B can take ~140–170s) isn't cut off mid-answer. Raise it only if you switch to a very large model and start hitting timeouts; there's rarely a reason to lower it. Set it **per query** in the **Timeout (s)** field, or **per corpus** in **Edit** → **Query Defaults** — same precedence as the other knobs (query field → corpus default → system default).

Understanding Citations

Every answer includes numbered citations — `[1]`, `[2]`, `[3]` — in the text. These refer to the source passages listed in the **References** panel on the right side of the screen.

What citations tell you: - Which document the information came from - Which page in that document

What to do when an answer seems wrong: Check the citations first. If the referenced chunks don't actually contain the claim — the model may be reasoning beyond its sources. Try increasing Top K, adjusting the Retrieval Mix toward Literal, or rephrasing to be more specific.

Uncited claims: if a sentence has no citation marker, the model is drawing on general context rather than a specific retrieved passage. Treat uncited claims with more caution, especially for specific numbers or facts.

Opening the source: click the citation chip in the References panel to see the full chunk text and open the source document.

Benchmark & Corpus Testing

The benchmark harness serves two purposes: - **Performance** — measures query latency per question - **Veracity** — validates answer quality against a questions file

How do I run a benchmark on the demo corpus?

```
ais_bench
```

Parameters (Top K, Temperature, Retrieval Mix, Score Threshold) are read automatically from corpus metadata — no flags needed for built-in corpora.

How do I benchmark a different corpus?

```
ais_bench --corpus sec_10k
ais_bench --corpus help
```

How do I test with a specific model or override settings?

```
ais_bench --corpus demo --model llama3.1:70b
ais_bench --corpus demo --alpha 0.3 --min-score 0.2
ais_bench --help   # all flags
```

What makes a benchmark question pass or fail? Three checks — all must pass: 1. All `keywords` appear in the answer (case-insensitive) 2. The answer includes at least one citation 3. The model doesn't hedge with "no information available" or similar phrases

How do I write my own test questions for a corpus? Create `benchmarks/<corpus_name>/<corpus_name>_questions.yaml`:

```
- topic: Your Topic
  questions:
    - id: unique_snake_case_id
      question: The exact question text sent to AIStudio.
      keywords: [term1, term2, term3]   # all must appear in the answer to pass
      notes: What a correct answer looks like — which document, what content.
```

Run `ais_bench --corpus <name>` — the questions file is auto-detected by corpus name.

Tips for good keywords: use 2–5 distinctive terms that prove the model retrieved the right content — specific concepts, entity names, or regulatory terms. Avoid generic words that would appear in any answer.

Where do reports go? `benchmarks/<corpus>/reports/` — timestamped `.md`, `.json`, and `.pdf` files. The `.md` shows pass/fail per question with latency, citations, and the answer text.

PDF reports aren't being generated — I see "PDF skipped"

`weasyprint` is required for PDF output and must be installed separately:

```
pip3 install weasyprint --break-system-packages
```

It's already in `requirements.txt` but not installed by default. Once installed, subsequent runs generate `.pdf` reports alongside `.md` and `.json`. The `⚠ PDF skipped` message is printed when weasyprint is missing — `.md` and `.json` reports are always generated regardless.

See `TUTORIAL.md` Module 5 (§5.7) for a step-by-step walkthrough.

Troubleshooting

UI shows "Error loading corpora" or "Ollama not running" on startup

The browser opened before the backend finished starting. Hard-refresh (`Cmd+Shift+R`). If that doesn't fix it, check the terminal for a Qdrant WAL error (see below).

Ollama version mismatch warning — "client version is X.X.X"

After a Homebrew update, Ollama's CLI binary may update while the background service still runs the old version. You'll see:

```
ollama version is 0.18.0
Warning: client version is 0.22.0
```

Fix:

```
brew services restart ollama
ais_stop
ais_start
```

Qdrant WAL lock — "Can't init WAL: Resource temporarily unavailable"

What it is: A corpus collection's write-ahead log was left locked from an unclean shutdown — force-quit terminal, power loss, or crash mid-write. Qdrant panics on the affected collection at startup. Other collections are unaffected.

How you'll know: Terminal shows the WAL error on `ais_start`, naming the specific collection. UI shows "Error loading corpora." Ingestion immediately fails.

Fix:

```
ais_stop
# Delete only the collection named in the panic message — e.g. aistudio_help
rm -rf ~/qdrant_storage/collections/aistudio_help
ais_start
```

Then re-ingest the corpus via the UI (Add button).

⚠️ Delete only the specific collection named in the error — not the entire `~/qdrant_storage/` folder. That would destroy all your corpora.

Prevention: Always stop with `ais_stop`. Never force-quit or close the terminal while AIStudio is running.

Ingestion summary shows wrong file or chunk count

Delete the corpus via the UI and re-ingest. The Qdrant collection may be out of sync with the uploads folder.

How do I rename a corpus? Use the **Rename** button in the corpus header in the UI. AIStudio renames the directory, updates the corpus metadata, and triggers a background re-index. You'll see a progress estimate for large corpora. Do not rename corpus folders manually on disk — use the UI only.

How do I rename a file inside a corpus? AIStudio records file names at ingest time. Renaming a file after ingestion breaks citation lookup — queries will still return results but the "Open ↗" link will fail to resolve.

Safe approach: delete the file from the corpus using the **Delete** button in the Inspect view, then re-upload it with the correct filename via **Add**. AIStudio will re-ingest only the changed file.

For architecture context, see docs/architecture_decisions.pdf. For getting started, see QUICKSTART.md.